

Relacionar teórica: análise semântica com ANTLR

Constantino Martins

- Relacionar teoria de análise semântica com ANTLR
- Perceber como atributos e ações aparecem no código
- Eliminar confusão entre parsing e execução
- Dar uma visão clara de como implementar semântica

Teoria: Ações Semânticas

- Revisão:
 - São executadas quando uma regra é aplicada
 - Exemplo:
 - $E \rightarrow E + T \{ E.val = E1.val + T.val \}$
 - Permitem calcular valores ou verificar tipos

Atributos no ANTLR

- `expr` returns `[int v]`
 - ✓ Atributos sintetizados são definidos com `returns`
- Acesso com `$v`, `$a.v`, `$b.v`
- Equivalente direto a `E.val`

Atributos herdados:

`decl`

`: t=type list[$t.name] ';' ;`

`;`

`list[String inheritedType]`

`: id=ID`

`{ types.put($id.getText(), inheritedType); }`

`inheritedType` → valor recebido do contexto

Correspondência: `L.type = T.type`

Neste exemplo `list` está simplificado

ANTLR: Regras com Labels

- `left=expr '+' right=expr`
- `left` corresponde ao operando esquerdo
- `right` corresponde ao operando lado direito
- Resolve problema de E1, E2 na teoria

Teoria $\rightarrow$ ANTLR

- $E.val \rightarrow \$v$
- $E1.val \rightarrow \$left.v$
- $T.val \rightarrow \$right.v$
- Mesma lógica, sintaxe diferente

Exemplo

- $E \rightarrow E + T \{ E.val = E1.val + T.val \}$
- $T \rightarrow \text{numero} \{ T.val = \text{numero.val} \}$
- Nota:
 - A gramática reconhece estrutura
 - A ação calcula o valor

ANTLR

expr returns [int v]

: a=expr '+' b=expr { \$v = \$a.v + \$b.v; }

| INT { \$v = Integer.valueOf(\$INT.getText()); }

- ANTLR implementa diretamente a teoria

Parsing vs Semântica

- ANTLR constrói parse tree
- Valida apenas a sintaxe
- Não calcula valores automaticamente
- Semântica tem de ser implementada

Implementar semântica

- ✓ Ações na gramática
- ✓ Listeners
- ✓ Visitors

São três formas de implementar semântica

Para calcular valores, Visitors são a abordagem mais natural fora da gramática

Exemplo

- Converter para ANTLR:
- $E \rightarrow E * T \{ E.val = E1.val * T.val \}$
- Usar returns, labels e ações

```
expr returns [int v]
: a=expr '*' b=expr
{
    $v = $a.v * $b.v;
}
| INT
{
    $v = Integer.valueOf($INT.getText());
}
;
```

```
MUL : '*' ;
INT : [0-9]+ ;
WS : [ \t\r\n]+ -> skip ;
```

Exercício

- Acrescentar a divisão

```
expr returns [int v]
: a=expr op=('*' | '/') b=expr
{
  if ($op.getText().equals("*"))
    $v = $a.v * $b.v;
  else
    $v = $a.v / $b.v;
}
| INT
{ $v = Integer.valueOf($INT.getText()); }
;
```

```
MUL : '*';
DIV : '/';
INT : [0-9]+ ;
WS : [ \t\r\n]+ -> skip ;
```

Exercício

- Acrescentar a divisão

```
expr returns [int v]
: a=expr op=(MUL | DIV) b=expr
{
  if ($op.getType() == MUL) {
    $v = $a.v * $b.v;
  } else {
    if ($b.v == 0) {
      System.err.println("Erro: divisão por zero");
      $v = 0; // ou outra estratégia
    } else {
      $v = $a.v / $b.v;
    }
  }
}
| INT
{
  $v = Integer.valueOf($INT.getText());
}
;
```

```
MUL : '*' ;
DIV : '/' ;
INT : [0-9]+ ;
WS : [ \t\r\n]+ -> skip ;
```

Exemplo 2

```
int a;  
float b;  
  
a = 3;  
b = 2.5;  
a = 2.5; // erro semântico
```

```
grammar TypeCheck;  
  
@parser::header {  
import java.util.*;  
}  
  
@parser::members {  
Map<String, String> types = new  
HashMap<>();  
}  
  
prog  
    : stat+ EOF  
    ;
```

Exercício 2

```
int a;  
float b;  
  
a = 3;  
b = 2.5;  
a = 2.5; // erro semântico
```

```
FLOAT : [0-9]+ '.' [0-9]+ ;  
INT   : [0-9]+ ;  
ID    : [a-zA-Z]+ ;  
WS    : [ \t\r\n]+ -> skip ;
```

```
stat  
  : decl  
  | assign  
  ;  
  
decl  
  : t=type id=ID ';' {  
    {  
      if (types.containsKey($id.getText())) {  
        System.err.println("Erro: variável já  
declarada: " + $id.getText());  
      } else {  
        types.put($id.getText(), $t.name);  
      }  
    }  
  }  
  ;
```

Exercício 2

```
int a;  
float b;  
  
a = 3;  
b = 2.5;  
a = 2.5; // erro semântico
```

```
FLOAT : [0-9]+ '.' [0-9]+ ;  
INT   : [0-9]+ ;  
ID    : [a-zA-Z]+ ;  
WS    : [ \t\r\n]+ -> skip ;
```

```
assign  
: id=ID '=' e=expr ';' {  
    String varName = $id.getText();  
  
    if (!types.containsKey(varName)) {  
        System.err.println("Erro: variável não declarada: " +  
varName);  
    } else if (!types.get(varName).equals($e.type)) {  
        System.err.println(  
            "Erro: tipos incompatíveis na atribuição a " + varName  
+  
            ". Esperado: " + types.get(varName) +  
            ", encontrado: " + $e.type  
        );  
    }  
}  
;
```

Exercício 2

```
int a;  
float b;  
  
a = 3;  
b = 2.5;  
a = 2.5; // erro semântico
```

```
FLOAT : [0-9]+ '.' [0-9]+ ;  
INT   : [0-9]+ ;  
ID    : [a-zA-Z]+ ;  
WS    : [ \t\r\n]+ -> skip ;
```

```
type returns [String name]  
: 'int' { $name = "int"; }  
| 'float' { $name = "float"; }  
;
```

```
expr returns [String type]  
: INT  
  { $type = "int"; }  
| FLOAT  
  { $type = "float"; }  
| id=ID  
  {  
    String varName = $id.getText();  
  
    if (!types.containsKey(varName)) {  
      System.err.println("Erro: variável não  
declarada: " + varName);  
      $type = "error";  
    } else {  
      $type = types.get(varName);  
    }  
  }  
;
```